

```

1 -----
2 -- Fixed Point Library Function
3 --
4 -- This file declares and defines fixed point functions, procedures, and
5 -- constants used in the Cordic implementation and testbench.
6 --
7 -- Revision History:
8 --   Date:           Author:   Description:
9 --   31 Jan 2024    Chris M.   Initial revision
10 -----
11
12 -- libraries
13 library ieee;
14 use std.textio.all;
15 use ieee.std_logic_1164.all;
16 use ieee.numeric_std.all;
17 use ieee.math_real.all;
18
19 -- Fixed point utility library declaration. Reused in test bench.
20 package FixedLib is
21
22 -- The number of integer and fractional bits in a Q3.18 fixed point
23 constant Q3_18_INTEGER_BITS    : integer := 4;
24 constant Q3_18_FRACTIONAL_BITS : integer := 18;
25
26 -- The number of integer and fractional bits in a Q1.14 fixed point
27 constant Q1_14_INTEGER_BITS    : integer := 2;
28 constant Q1_14_FRACTIONAL_BITS : integer := 14;
29
30 -- Conversion functions. Overloaded to accept signed integers or std_logic_vectors.
31 function RealToQ3_18 (real_val : real) return std_logic_vector;
32 function RealToQ3_18 (real_val : real) return integer;
33
34 function RealToQ1_14 (real_val : real) return std_logic_vector;
35 function RealToQ1_14 (real_val : real) return integer;
36
37 function Q1_14ToReal (fixed_val : std_logic_vector) return real;
38 function Q1_14ToReal (fixed_val : integer)           return real;
39
40 function Q3_18ToReal (fixed_val : std_logic_vector) return real;
41 function Q3_18ToReal (fixed_val : integer)           return real;
42
43 -- Get the min and max values for arbitrary fixed points.
44 function GetUpperRange (IntegerBits : positive; FractionalBits : positive) return real;
45 function GetLowerRange (IntegerBits : positive; FractionalBits : positive) return real;
46 function GetPrecision  (IntegerBits : positive; FractionalBits : positive) return real;
47
48 procedure PrintQ1_14_Fixed( signal  fixed   : in    std_logic_vector(15 downto 0);
49                             variable my_line : inout line);
50
51 procedure PrintQ3_18_Fixed( signal  fixed   : in    std_logic_vector(21 downto 0);
52                             variable my_line : inout line);
53
54 -- Convert a Q3.18 fixed point to a Q1.14 fixed point.
55 function Q3_18_To_Q1_14(slv : std_logic_vector(21 downto 0)) return std_logic_vector;
56
57 end FixedLib;
58

```

```

59 package body FixedLib is
60
61 -----
62 -- Convert real numbers to Q3_18 fixed points
63 function RealToQ3_18 (real_val : real) return std_logic_vector is
64     variable result : std_logic_vector(21 downto 0);
65     variable tmp     : real;
66 begin
67     -- Round towards zero.
68     if (real_val < 0.0) then
69         tmp := ceil(real_val * (2.0 ** Q3_18_FRACTIONAL_BITS));
70     else
71         tmp := floor(real_val * (2.0 ** Q3_18_FRACTIONAL_BITS));
72     end if;
73     result := std_logic_vector(to_signed(integer(tmp), 22));
74     return result;
75 end function;
76 -----
77
78 -----
79 function RealToQ3_18 (real_val : real) return integer is
80     variable slv : std_logic_vector(21 downto 0);
81 begin
82     slv := RealToQ3_18(real_val);
83     return to_integer(signed(slv));
84 end function;
85 -----
86
87 -----
88 function RealToQ1_14 (real_val : real) return std_logic_vector is
89     variable result : std_logic_vector(15 downto 0);
90     variable tmp     : real;
91 begin
92     if (real_val < 0.0) then
93         tmp := ceil(real_val * (2.0 ** Q1_14_FRACTIONAL_BITS));
94     else
95         tmp := floor(real_val * (2.0 ** Q1_14_FRACTIONAL_BITS));
96     end if;
97     result := std_logic_vector(to_signed(integer(tmp), 16));
98     return result;
99 end function;
100 -----
101
102 -----
103 function RealToQ1_14 (real_val : real) return integer is
104     variable slv : std_logic_vector(15 downto 0);
105 begin
106     slv := RealToQ1_14(real_val);
107     return to_integer(signed(slv));
108 end function;
109 -----
110
111 -----
112 function Q1_14ToReal (fixed_val : std_logic_vector) return real is
113     variable int_val : integer;
114     variable result   : real;
115 begin
116     int_val := to_integer(signed(fixed_val));

```

```

117     result := real(int_val) / (2.0 ** 14);
118     return result;
119 end function;
120 -----
121
122 -----
123 function Q1_14ToReal (fixed_val : integer) return real is
124 begin
125     return Q1_14ToReal (std_logic_vector(to_signed(fixed_val, 16)));
126 end function;
127 -----
128
129 -----
130 function Q3_18ToReal (fixed_val : std_logic_vector) return real is
131     variable int_val : integer;
132     variable result : real;
133 begin
134     int_val := to_integer(signed(fixed_val));
135     result := real(int_val) / (2.0 ** 18);
136     return result;
137 end function;
138 -----
139
140 -----
141 function Q3_18ToReal (fixed_val : integer) return real is
142     variable slv : std_logic_vector(21 downto 0);
143 begin
144     slv := std_logic_vector(to_signed(fixed_val, 22));
145     return Q3_18ToReal(slv);
146 end function;
147 -----
148
149 -----
150 function GetUpperRange(IntegerBits : positive; FractionalBits : positive) return real is
151     variable TmpDividend : integer;
152     variable TmpDivisor : integer;
153     variable TmpReal : real;
154 begin
155     TmpDivisor := (2 ** (IntegerBits + FractionalBits - 1)) - 1;
156     TmpDividend := (2 ** FractionalBits);
157     TmpReal := real(TmpDivisor) / real(TmpDivisor);
158     return TmpReal;
159 end function;
160 -----
161
162 -----
163 -----
164 function GetLowerRange(IntegerBits : positive; FractionalBits : positive) return real is
165     variable TmpDividend : integer;
166     variable TmpDivisor : integer;
167     variable TmpReal : real;
168 begin
169     TmpDividend := -(2 ** (IntegerBits + FractionalBits - 1));
170     TmpDivisor := (2 ** FractionalBits);
171     TmpReal := real(TmpDividend) / real(TmpDivisor);
172     return TmpReal;
173 end function;
174 -----

```

```
175
176
177 -----
178 function GetPrecision(IntegerBits : positive; FractionalBits : positive) return real is
179 begin
180     return (1.0 / (real (2 ** FractionalBits) ));
181 end function;
182 -----
183
184
185 -----
186 -- Print Q1_14 fixed point numbers
187 procedure PrintQ1_14_Fixed(
188     signal fixed      : in      std_logic_vector(15 downto 0);
189     variable my_line  : inout line) is
190     variable int_val  : integer;
191     variable real_val : real;
192 begin
193     int_val := to_integer(signed(fixed));
194     real_val := real(int_val) / (2.0 ** 14);
195     write(my_line, HT);
196     write(my_line, real_val, LEFT, 12, 9);
197     write(my_line, string'(HT & LF));
198 end procedure;
199 -----
200
201 -----
202 -- Print Q3_18 fixed point numbers
203 procedure PrintQ3_18_Fixed(
204     signal fixed      : in      std_logic_vector(21 downto 0);
205     variable my_line  : inout line) is
206     variable int_val  : integer;
207     variable real_val : real;
208 begin
209     int_val := to_integer(signed(fixed));
210     real_val := real(int_val) / (2.0 ** 18);
211     write(my_line, HT);
212     write(my_line, real_val, LEFT, 12, 9);
213     write(my_line, HT);
214 end procedure;
215 -----
216
217 -----
218 -- Convert a Q3.18 fixed point to a Q1.14 fixed point.
219 function Q3_18_To_Q1_14(slv : std_logic_vector(21 downto 0)) return std_logic_vector is
220 begin
221     return slv(21) & slv(18) & slv(17 downto 4);
222 end function;
223 -----
224
225 end package body FixedLib;
226
227
```